WARM-UP · W-A3-01 · SQL AGGREGATION 2 MIN

COUNT oder SUM?

Gib für die Fragen „Wie viele Bons gab es?“ und „Wie viele Crêpes wurden verkauft?“ jeweils die passende Aggregatfunktion an und begründe.

Kompetenz K25 · Falle P4

WARM-UP · W-A3-02 · SQL AGGREGATION 3 MIN

Regel-Check

Prüfe, ob die Abfrage die GROUP-BY-Regel erfüllt, und korrigiere sie.

```
SELECT Stand.Name, Bon.Zahlart, COUNT(*) AS Bons
FROM Stand, Bon
WHERE Stand.StandNr = Bon.StandNr
GROUP BY Stand.Name
```

Kompetenz K26 · Falle P3

WARM-UP · W-A3-03 · ABFRAGE ANALYSIEREN 2 MIN

Fächer füllen

Die Bons 503 bis 510 haben diese Zahlarten: 503 Karte · 504 bar · 505 bar · 506 Karte · 507 bar · 508 Karte · 509 bar · 510 bar. Gib das Ergebnis der Abfrage für diese acht Bons an.

```
SELECT Zahlart, COUNT(*) AS Bons
FROM Bon
WHERE BonNr BETWEEN 503 AND 510
GROUP BY Zahlart
```

Kompetenz K26, K29

WARM-UP · W-A3-04 · SQL AGGREGATION 3 MIN

Zwei Lotta Bödeker

Am Stadtfest haben zwei Helferinnen namens Lotta Bödeker kassiert. Begründe, warum für den Umsatz je Person nach `Helfer.HelferNr` und nicht nur nach Vor- und Nachnamen gruppiert wird.

Kompetenz K26 · Falle P3

WARM-UP · W-A3-05 · SQL AGGREGATION 2 MIN

WHERE oder HAVING?

Entscheide für jede Bedingung, ob sie in WHERE oder HAVING gehört, und nenne je einen Grund: `Bon.Zahlart = 'Karte'` · `COUNT(*) > 20` · `Produkt.Preis < 3` · `SUM(Position.Menge) >= 50`.

Kompetenz K27 · Falle P1

WARM-UP · W-A3-06 · SQL AGGREGATION 2 MIN

Reihenfolge

Notiere die Auswertungsreihenfolge der Klauseln SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT. Erkläre damit, warum in HAVING kein Alias aus dem SELECT verwendet wird, in ORDER BY aber schon.

Kompetenz K27 · Falle P1

WARM-UP · W-A3-07 · ABFRAGE ANALYSIEREN 3 MIN

Ein Satz Sachzusammenhang

Beschreibe in **einem** Satz im Sachzusammenhang, welche Information die Abfrage liefert.

```
SELECT Produkt.ProduktNr, Produkt.Bezeichnung, SUM(Position.Menge) AS Stueck
FROM Produkt, Position, Bon
WHERE Produkt.ProduktNr = Position.ProduktNr
  AND Position.BonNr = Bon.BonNr
  AND Bon.Zeitpunkt LIKE '2026-08-28%'
GROUP BY Produkt.ProduktNr, Produkt.Bezeichnung
HAVING SUM(Position.Menge) > 30
ORDER BY Stueck DESC
```

Kompetenz K28 · Falle P10

WARM-UP · W-A3-08 · SQL AGGREGATION 3 MIN

Platz drei

Angenommen, die Stände auf Platz 3 und 4 haben genau denselben Umsatz. Begründe, ob die Abfrage dann „die drei umsatzstärksten Stände" eindeutig liefert.

```
SELECT Stand.StandNr, Stand.Name, SUM(Position.Menge * Produkt.Preis) AS Umsatz
FROM Stand, Bon, Position, Produkt
WHERE Stand.StandNr = Bon.StandNr
  AND Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
GROUP BY Stand.StandNr, Stand.Name
ORDER BY Umsatz DESC
LIMIT 3
```

Kompetenz K22 · Falle P6

Lösungen der Warm-up-Karten

NUR LEHRKRAFT – dieser Teil fehlt in der Schülerfassung.

W-A3-01 · COUNT oder SUM?

„Anzahl Bons“: `COUNT(*)` über Bon – jeder Bon ist eine Zeile. „Verkaufte Crêpes“: `SUM(Position.Menge)` – eine Position kann mehrere Crêpes enthalten; `COUNT(*)` würde nur Positionen zählen.

W-A3-02 · Regel-Check

`Bon.Zahlart` ist weder aggregiert noch gruppiert – Verstoß. Korrektur: nach dem Schlüssel des Standes, dem Namen **und** der Zahlart gruppieren.

```
SELECT Stand.Name, Bon.Zahlart, COUNT(*) AS Bons
FROM Stand, Bon
WHERE Stand.StandNr = Bon.StandNr
GROUP BY Stand.StandNr, Stand.Name, Bon.Zahlart
```

W-A3-03 · Fächer füllen

Zwei Fächer, zwei Zeilen: `bar | 5` und `Karte | 3`.

```
SELECT Zahlart, COUNT(*) AS Bons
FROM Bon
WHERE BonNr BETWEEN 503 AND 510
GROUP BY Zahlart
```

W-A3-04 · Zwei Lotta Bödeker

Namen sind nicht eindeutig, der Schlüssel HelferNr schon. Gruppiert man nur nach Vor- und Nachnamen, landen beide Personen in **einem** Fach und ihre Umsätze werden addiert (68,00 € + 67,50 € = 135,50 €). Deshalb: `GROUP BY Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname`.

```
SELECT Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname, SUM(Position.Menge * Produkt.Preis) AS
Umsatz
FROM Helfer, Bon, Position, Produkt
WHERE Helfer.HelferNr = Bon.HelferNr
  AND Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
  AND Helfer.Nachname = 'Bödeker'
GROUP BY Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname
```

W-A3-05 · WHERE oder HAVING?

WHERE: `Bon.Zahlart = 'Karte'` und `Produkt.Preis < 3` – sie prüfen einzelne Zeilen vor dem Gruppieren.
HAVING: `COUNT(*) > 20` und `SUM(Position.Menge) >= 50` – sie enthalten ein Aggregat und prüfen ganze Gruppen.

W-A3-06 · Reihenfolge

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. Der Alias entsteht erst im SELECT; HAVING wird vorher ausgewertet und wiederholt deshalb das Aggregat (EH). ORDER BY kommt nach SELECT und darf den Alias nutzen.

W-A3-07 · Ein Satz Sachzusammenhang

Die Abfrage liefert für jedes Produkt (Nummer und Bezeichnung), wie viele Stück davon am Festfreitag verkauft wurden – aber nur Produkte mit mehr als 30 verkauften Stück, die meistverkauften zuerst.

```
SELECT Produkt.ProduktNr, Produkt.Bezeichnung, SUM(Position.Menge) AS Stueck
FROM Produkt, Position, Bon
WHERE Produkt.ProduktNr = Position.ProduktNr
  AND Position.BonNr = Bon.BonNr
  AND Bon.Zeitpunkt LIKE '2026-08-28%'
GROUP BY Produkt.ProduktNr, Produkt.Bezeichnung
HAVING SUM(Position.Menge) > 30
ORDER BY Stueck DESC
```

W-A3-08 · Platz drei

Nein. `LIMIT 3` schneidet nach der dritten Zeile ab; welcher der beiden gleich starken Stände auf Platz 3 landet, ist Zufall. Eindeutig wird die Frage z. B. als „alle Stände mit mindestens so viel Umsatz wie Platz 3" oder mit einer begründeten zweiten Sortierstufe.